

# Tugas Penerapan Filter Spasial

Penulis: Kenny Gabriel Manurung  
Dosen: Arta Kusuma Hernanda  
NRP : 5024241022  
Nama Kelas: Pengolahan Citra dan Video (A)

## Kode yang Digunakan

```
import cv2
import numpy as np

# READ IMAGE

image = cv2.imread("rb.png")

if image is None:
    print("Gambar tidak ditemukan!")
    exit()

print("Gambar berhasil dibaca")
print("Ukuran gambar:", image.shape)

# FUNGSI SPATIAL FILTER

def spatial_filter(image, kernel):
    kernel = np.array(kernel, dtype=np.float32)

    kernel_height, kernel_width = kernel.shape

    pad_height = kernel_height // 2
    pad_width = kernel_width // 2

    # Padding agar ukuran output tetap sama dengan gambar asli
    padded = np.pad(
        image,
        ((pad_height, pad_height),
         (pad_width, pad_width),
         (0, 0)),
        mode="edge"
    )

    output = np.zeros_like(image, dtype=np.float32)
```

```

height, width, channels = image.shape

for y in range(height):
    for x in range(width):

        for c in range(channels):

            region = padded[
                y:y + kernel_height,
                x:x + kernel_width,
                c
            ]

            output[y, x, c] = np.sum(region * kernel)

output = np.clip(output, 0, 255)

return output.astype(np.uint8)

```

```

# 1. MEAN FILTER / AVERAGE FILTER
#     Blur / Smoothing

```

```

mean_kernel = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
], dtype=np.float32) / 9

mean_filter = spatial_filter(image, mean_kernel)

```

```

# 2. GAUSSIAN BLUR

```

```

gaussian_kernel = np.array([
    [1, 4, 6, 4, 1],
    [4, 16, 24, 16, 4],
    [6, 24, 36, 24, 6],
    [4, 16, 24, 16, 4],
    [1, 4, 6, 4, 1]
], dtype=np.float32) / 256

gaussian_filter = spatial_filter(image, gaussian_kernel)

```

```

# 3. MEDIAN FILTER
#     Noise Reduction

```

```

def median_filter(image, kernel_size=3):

    pad = kernel_size // 2

    padded = np.pad(
        image,

```

```

        ((pad, pad),
         (pad, pad),
         (0, 0)),
        mode="edge"
    )

    output = np.zeros_like(image)

    height, width, channels = image.shape

    for y in range(height):
        for x in range(width):

            for c in range(channels):

                region = padded[
                    y:y + kernel_size,
                    x:x + kernel_size,
                    c
                ]

                output[y, x, c] = np.median(region)

    return output.astype(np.uint8)

```

```

median_filter_result = median_filter(image, 3)

```

#### # 4. *SHARPENING*

```

sharpen_kernel = np.array([
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
], dtype=np.float32)

sharpen_filter = spatial_filter(image, sharpen_kernel)

```

#### # 5. *EMBOSS*

```

emboss_kernel = np.array([
    [-2, -1, 0],
    [-1, 1, 1],
    [0, 1, 2]
], dtype=np.float32)

emboss_filter = spatial_filter(image, emboss_kernel)

```

#### # 6. *SOBEL X*

# Mendeteksi perubahan intensitas arah horizontal

```

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

```

```

sobel_x_kernel = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)

def spatial_filter_gray(image, kernel):

    kernel = np.array(kernel, dtype=np.float32)

    kernel_height, kernel_width = kernel.shape

    pad_height = kernel_height // 2
    pad_width = kernel_width // 2

    padded = np.pad(
        image,
        ((pad_height, pad_height),
         (pad_width, pad_width)),
        mode="edge"
    )

    output = np.zeros_like(image, dtype=np.float32)

    height, width = image.shape

    for y in range(height):
        for x in range(width):

            region = padded[
                y:y + kernel_height,
                x:x + kernel_width
            ]

            output[y, x] = np.sum(region * kernel)

    output = np.abs(output)

    output = np.clip(output, 0, 255)

    return output.astype(np.uint8)

sobel_x = spatial_filter_gray(gray, sobel_x_kernel)

# 7. SOBEL Y
# Mendeteksi perubahan intensitas arah vertikal

sobel_y_kernel = np.array([
    [-1, -2, -1],
    [0, 0, 0],
    [1, 2, 1]
], dtype=np.float32)

sobel_y = spatial_filter_gray(gray, sobel_y_kernel)

```

```

# 8. LAPLACIAN
#    Edge Detection

laplacian_kernel = np.array([
    [0, 1, 0],
    [1, -4, 1],
    [0, 1, 0]
], dtype=np.float32)

laplacian_filter = spatial_filter_gray(gray, laplacian_kernel)

# MENAMPILKAN HASIL

cv2.imshow("Original", image)
cv2.imshow("Mean_Filter_Blur", mean_filter)
cv2.imshow("Gaussian_Blur", gaussian_filter)
cv2.imshow("Median_Filter_Noise_Reduction", median_filter_result)
cv2.imshow("Sharpening", sharpen_filter)
cv2.imshow("Emboss", emboss_filter)
cv2.imshow("Sobel_X_Edge_Detection", sobel_x)
cv2.imshow("Sobel_Y_Edge_Detection", sobel_y)
cv2.imshow("Laplacian_Edge_Detection", laplacian_filter)

cv2.waitKey(0)
cv2.destroyAllWindows()

```